DESENVOLVIMENTO MOBILE · MÓDULO 01 · TRILHA FLUTTER · NÍVEL INICIANTE

Ambiente de Desenvolvimento

Flutter SDK, editores, emuladores e celular real, estrutura do projeto, pubspec, hot reload, DevTools e pacotes

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o que é o Flutter e como ele chega ao Android, ao iOS, à web e ao desktop.
- Instalar o Flutter SDK e diagnosticar o ambiente com `flutter doctor`.
- Executar apps no emulador, no simulador iOS e no celular real.
- Entender a estrutura de um projeto Flutter e o arquivo `pubspec.yaml`.
- Usar *hot reload*, os modos de execução e o Flutter DevTools.
- Encontrar, avaliar e adicionar pacotes do **pub.dev**.

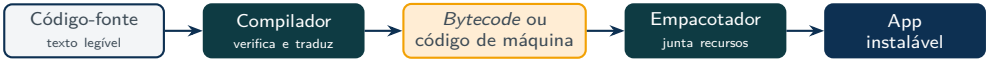
Roteiro

Neste módulo

- Fundamentos: ferramentas de desenvolvimento
- Flutter e Dart
- Instalação
- Estrutura do projeto
- Executando e depurando
- Pacotes e comandos
- Fechamento

Fundamentos: ferramentas de desenvolvimento

Do código-fonte ao programa



Compilador e interpretador

O **compilador** traduz o programa inteiro antes da execução e aponta erros cedo. O **interpretador** lê e executa o código aos poucos, durante a execução.

Máquina virtual

Programa que executa um *bytecode* intermediário em qualquer processador: a JVM e o ART (Kotlin) e a VM do Dart.

JIT e AOT

JIT: compila enquanto o programa roda; permite trocar código em execução (*hot reload*). **AOT**: compila tudo antes de distribuir; inicia rápido e roda com desempenho previsível.

Dica

As duas trilhas usam os dois: JIT para desenvolver com agilidade, AOT para o app que chega ao usuário (Módulo 15).

SDK, IDE e cadeia de ferramentas

SDK — *Software Development Kit*

Pacote com bibliotecas, compiladores, ferramentas de linha de comando, documentação e imagens de emulador para desenvolver para uma plataforma.

IDE — ambiente integrado

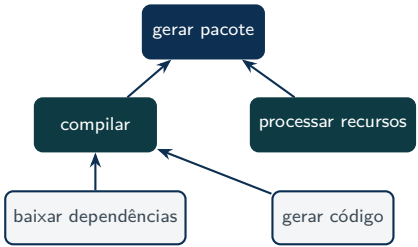
Editor que reúne escrita de código, compilação, execução, depuração, análise e integração com Git.

Analogia

O SDK é a caixa de ferramentas; a IDE é a bancada organizada onde você trabalha com elas.

	Trilha Kotlin	Trilha Flutter
Linguagem	Kotlin (sobre o JDK)	Dart (Dart SDK)
SDK da plataforma	Android SDK	Flutter SDK + Android SDK + Xcode
IDE	Android Studio	VS Code ou Android Studio
Build	Gradle	`flutter` CLI (usa Gradle e Xcode)
Aparelho	ADB	ADB e `flutter devices`
Pacotes	Maven Central, Google Maven	pub.dev

Sistemas de *build* e dependências



Grafo de tarefas: cada uma só roda depois das que ela usa.

Sistema de *build*

Ferramenta que transforma o projeto em app executando **tarefas** organizadas num grafo de dependências. Só refaz o que mudou (*build* incremental) e guarda resultados em cache.

Dependência transitiva

A biblioteca que você usa depende de outras, que dependem de outras. O gerenciador resolve a árvore inteira e escolhe versões compatíveis.

Atenção

Fixe as versões com arquivos de trava (catálogo de versões, `pubspec.lock`) para que todos na equipe compilem o mesmo app.

Emuladores, simuladores e aparelhos reais

	Vantagem	Limite
Emulador Android	vários tamanhos e versões, GPS e sensores simulados	sem Bluetooth, NFC e USB reais
Simulador iOS	rápido, integrado ao Xcode	roda código do Mac; não imita o hardware
Aparelho real	desempenho e hardware verdadeiros	cada modelo exige um aparelho

Emulador e simulador

O **emulador** imita o aparelho, inclusive o processador e o sistema completo. O **simulador** apenas reproduz o comportamento do sistema, executando o app compilado para o computador.

Por que o emulador exige virtualização

O emulador Android roda uma imagem completa do sistema numa máquina virtual. Com aceleração por hardware (Intel VT-x, AMD-V, KVM no Linux, Hypervisor no macOS), as instruções rodam direto no processador e o emulador fica rápido.

Depuração

Depurador

Ferramenta que pausa o programa num ponto escolhido (*breakpoint*) para inspecionar variáveis, a pilha de chamadas e executar passo a passo.

Comando	O que faz
Step over	executa a linha e para na próxima
Step into	entra na função chamada
Step out	termina a função atual e volta
Resume	continua até o próximo *breakpoint*

Método científico da depuração

1. Reproduza o problema de forma confiável.
2. Formule uma hipótese sobre a causa.
3. Teste a hipótese (*breakpoint*, *log*, teste automatizado).
4. Corrija e escreva um teste que impeça a volta do defeito.

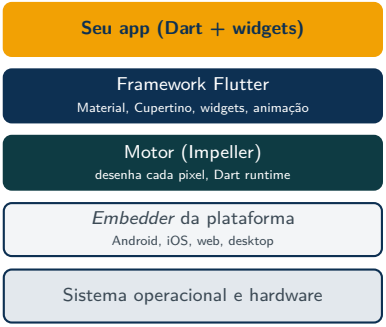
Dica

Leia a mensagem de erro e a pilha de chamadas inteiras: a primeira linha do **seu** código costuma apontar a causa.

Flutter e Dart

O que é o Flutter

- **Flutter**: kit de desenvolvimento de interfaces do Google, de código aberto.
- **Um único código** em **Dart** gera apps para Android, iOS, web, Windows, macOS e Linux.
- A interface é feita de **widgets** desenhados pelo próprio motor do Flutter, idênticos em todas as plataformas.
- Compilado para código nativo (AOT) em release: desempenho próximo ao nativo.
- *Hot reload*: mudanças aparecem no aparelho em menos de um segundo.



Por que Dart?

JIT no desenvolvimento (*hot reload*), AOT no release (desempenho), *null safety* e sintaxe familiar a quem conhece Java, C# ou JavaScript.

Instalação

Instalando o Flutter SDK

1. Instale o **VS Code** e a extensão **Flutter**; o comando *Flutter: New Project* oferece baixar o SDK.
2. Ou baixe o SDK no site oficial e ponha `flutter/bin` no `PATH`.
3. Instale o **Android Studio** (Android SDK e emulador).
4. `flutter doctor --android-licenses`.

iOS

Compilar para iOS exige **macOS** com Xcode. No Windows e no Linux, teste no Android, na web e no desktop.

flutter doctor (saída resumida)

```
$ flutter doctor
[ok] Flutter (Channel stable)
[ok] Android toolchain - Android SDK
[ x] Xcode (somente no macOS)
[ok] Chrome - develop for the web
[ok] Android Studio
[ok] VS Code
[ok] Connected device (2 available)
[ok] Network resources
```

Dica

`flutter doctor -v` mostra o que falta corrigir. Use o canal **stable**; `flutter upgrade` atualiza o SDK.

Onde executar o app

Destino	Como
Emulador Android	Device Manager do Android Studio
Celular Android	opções do desenvolvedor + depuração USB ou Wi-Fi
Simulador iOS	Xcode (somente macOS)
iPhone real	Xcode + conta Apple (gratuita para testes)
Navegador	Chrome, sem configuração extra
Desktop	Windows, macOS ou Linux, com as ferramentas de C++ do sistema

Hardware real

Bluetooth, NFC, câmera de verdade e sensores (Módulo 13) exigem **celular físico**.

Dispositivos pelo terminal

```
flutter devices              # lista destinos
flutter emulators            # lista AVDs
flutter emulators --launch Pixel_8
flutter run                  # escolhe o destino
flutter run -d chrome        # roda na web
flutter run -d emulator-5554
```

Celular Android

Ative as **Opções do desenvolvedor** (toque 7 vezes em *Número da versão*) e a **Depuração USB**. O mesmo vale para a trilha Kotlin (Módulo 01).

Estrutura do projeto

Criando o projeto

Criando pelo terminal

```
flutter create --org br.edu.ifma \
  --platforms android,ios,web labmonitor
cd labmonitor
flutter run
```

labmonitor/

```
labmonitor/
|-- lib/
|   `-- main.dart          ponto de entrada
|-- test/
|   `-- widget_test.dart   testes
|-- android/               projeto Android (Gradle)
|-- ios/                   projeto iOS (Xcode)
|-- web/
|-- pubspec.yaml           dependências e recursos
|-- pubspec.lock           versões exatas
`-- analysis_options.yaml  regras de análise
```

-org

Define o identificador do app: `br.edu.ifma.labmonitor`. Ele vira o `applicationId` no Android e o *bundle identifier* no iOS, e não pode mudar depois de publicado.

- Quase todo o seu código fica em `lib/`.
- `android/` e `ios/` são projetos nativos completos: você mexe neles para permissões, ícones e assinatura.
- `pubspec.lock` vai para o Git; garante as mesmas versões para toda a equipe.

pubspec.yaml

pubspec.yaml

```yaml
name: labmonitor
version: 1.0.0+1          # nome+número do build

environment:
  sdk: ^3.9.0

dependencies:
  flutter:
    sdk: flutter
  go_router: ^16.0.0      # exemplo de pacote

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  assets: [assets/imagens/]
```

- `version`: `1.0.0` é o nome exibido; `+1` é o número do build (`versionCode` no Android).
- `dependencies`: pacotes usados pelo app.
- `dev_dependencies`: só para desenvolvimento e testes.
- `assets`: imagens e arquivos empacotados (as versões do exemplo são ilustrativas).

Indentação

YAML usa espaços para definir a hierarquia. Um espaço a mais ou a menos quebra o arquivo.

lib/main.dart: o primeiro app

lib/main.dart

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const LabMonitorApp());
}

class LabMonitorApp extends StatelessWidget {
  const LabMonitorApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'LabMonitor',
      theme: ThemeData(
        colorSchemeSeed: const Color(0xFF0D3052)),
      home: Scaffold(
        appBar: AppBar(title: const Text('LabMonitor')),
        body: const Center(child: Text('Olá, laboratório!')),
      ),
    );
  }
}
```

- `main()` chama `runApp` com o widget raiz.
- `StatelessWidget`: widget sem estado próprio; descreve a tela no método `build`.
- `MaterialApp`: tema, rotas e localização do app.
- `Scaffold`: estrutura de tela com barra superior e corpo.
- `const`: widget criado em tempo de compilação, sem custo de reconstrução.

Executando e depurando

Hot reload e modos de execução

Ação	Tecla	Efeito
Hot reload	`r`	injeta o código novo e mantém o estado
Hot restart	`R`	reinicia o app, perde o estado
Full restart	parar e rodar	necessário após mudar código nativo ou o `pubspec`

Dica

No VS Code, salvar o arquivo já dispara o *hot reload*.

Modo	Uso
debug	desenvolvimento: JIT, *hot reload*, asserções, mais lento
profile	medir desempenho num celular real (`flutter run --profile`)
release	publicação: AOT, otimizado, sem ferramentas de depuração

Atenção

Nunca avalie desempenho em modo debug: ele pode ser várias vezes mais lento que o release.

Flutter DevTools e logs

DevTools

Ferramentas no navegador ou no editor:

- **Inspector**: árvore de widgets e depurador de layout.
- **Performance**: quadros lentos (*jank*).
- **CPU** e **Memory**: perfis e vazamentos.
- **Network**: requisições HTTP do app.
- **Logging**: mensagens e erros.

Registrando logs

```dart
import 'dart:developer' as dev;

void lerSensor(double valor) {
  debugPrint('Leitura: $valor');
  if (valor > 80) {
    dev.log('Temperatura alta: $valor',
        name: 'LabMonitor', level: 900);
  }
}
```

Depurador

No VS Code: *breakpoint* na margem e **F5** para depurar.

Atenção

Nunca registre senhas, tokens ou dados pessoais.

Pacotes e comandos

Pacotes do pub.dev

Gerenciando dependências

```
flutter pub add go_router      # adiciona
flutter pub add --dev mocktail
flutter pub remove go_router
flutter pub get                # baixa tudo
flutter pub outdated           # o que tem versão nova
flutter pub upgrade
```

Versões

`^2.1.0` aceita qualquer `2.x.y` a partir de `2.1.0`, mas não `3.0.0`, que pode quebrar a compatibilidade (versionamento semântico).

Como escolher um pacote

- **Publisher verificado** (ex.: `flutter.dev`, `firebase.google.com`).
- **Pub points** altos e atualizações recentes.
- Plataformas suportadas (Android, iOS, web...).
- **Licença** compatível com o seu uso.

Atenção

Pacote é código de terceiros no seu app: prefira poucos e bem mantidos.

Comandos do dia a dia

Comando	Para que serve
`flutter doctor`	diagnostica o ambiente
`flutter create`	cria um projeto
`flutter run`	compila, instala e executa com *hot reload*
`flutter analyze`	análise estática (erros e más práticas)
`dart format .`	formata todo o código
`flutter test`	roda os testes (Módulo 14)
`flutter build apk` / `appbundle` / `ipa` / `web`	gera os pacotes de publicação (Módulo 15)
`flutter clean`	apaga a pasta `build/` quando algo "estranho" acontecer
`dart run build_runner build`	executa geradores de código (JSON, Drift, Riverpod)

Git

O `.gitignore` gerado pelo `flutter create` já exclui `build/` e `.dart_tool/`. Faça o primeiro commit e publique no GitHub (Git em detalhes no Módulo 03).

Fechamento

Resumo

Resumo

- Flutter desenha a interface com o próprio motor; um código Dart atende Android, iOS, web e desktop.
- `flutter doctor` confere o ambiente; iOS exige macOS e Xcode.
- O código vive em `lib/`; `pubspec.yaml` declara versão, dependências e recursos.
- *Hot reload* (`r`) mantém o estado; *hot restart* (`R`) reinicia.
- Meça desempenho em modo **profile**; publique em **release**.
- Escolha pacotes do pub.dev por publisher, manutenção, plataformas e licença.

Glossário

Termo	Significado
SDK	conjunto de bibliotecas e ferramentas para desenvolver para uma plataforma
Flutter SDK	*framework*, motor, Dart SDK e a ferramenta `flutter`
`flutter doctor`	diagnóstico do ambiente: SDKs, licenças, aparelhos
`pubspec.yaml`	arquivo com nome, versão, dependências e recursos do projeto
pub.dev	repositório oficial de pacotes Dart e Flutter
Hot reload	injeta o código alterado no app em execução mantendo o estado
Hot restart	reinicia o app com o código novo, perdendo o estado
DevTools	ferramentas de inspeção, desempenho, memória e rede do Flutter
Breakpoint	ponto em que o depurador pausa a execução

Questões de revisão

1. Qual a diferença entre JIT e AOT? Como o Flutter usa cada um?
2. Por que o *hot reload* é possível em modo *debug* e não em *release*?
3. O que o `flutter doctor` verifica e por que rodá-lo depois de instalar?
4. Para que serve o `pubspec.lock` e por que ele vai para o Git?
5. Explique a diferença entre emulador Android e simulador iOS.
6. Cite duas situações em que é preciso um aparelho real.
7. Quando usar *hot restart* em vez de *hot reload*?
8. Descreva o método para depurar um app que fecha ao tocar num botão.

Exercícios

Exercício 1 — Ambiente

Rode `flutter doctor -v`, corrija todos os itens possíveis no seu sistema e execute o app padrão no emulador, no celular e no Chrome.

Exercício 2 — Hot reload

Mude a cor-semente do tema e o texto da tela com o app rodando. Compare *hot reload* e *hot restart* usando o contador do app padrão.

Exercício 3 — Pacotes

Pesquise no pub.dev três pacotes para gráficos. Compare publisher, pub points, plataformas e licença e justifique qual usaria.

LabMonitor — Etapa 1

Crie o projeto `labmonitor` (`-org br.edu.ifma`), troque o texto por "LabMonitor — DEE/IFMA", adicione um `debugPrint` no `main` e publique o repositório no GitHub.

Para aprofundar

- Instalação: `https://docs.flutter.dev/get-started`
- Arquitetura do Flutter: `https://docs.flutter.dev/resources/architectural-overview`
- Hot reload: `https://docs.flutter.dev/tools/hot-reload`
- DevTools: `https://docs.flutter.dev/tools/devtools`
- Usando pacotes: `https://docs.flutter.dev/packages-and-plugins/using-packages`
- Repositório de pacotes: `https://pub.dev`

Respostas das questões de revisão

1. JIT compila durante a execução; AOT, antes. O Flutter usa JIT em *debug* (para o *hot reload*) e AOT em *profile* e *release* (código ARM rápido).
2. O *hot reload* depende da VM do Dart com JIT, que aceita código novo em execução; em *release* o código já foi compilado AOT.
3. Instalação do Flutter, Android SDK e licenças, Xcode (no macOS), IDE e aparelhos conectados; aponta o que falta configurar.
4. Registra as versões exatas resolvidas de todas as dependências, para que todos compilem o mesmo app.
5. O emulador Android imita o aparelho e roda o sistema completo numa máquina virtual; o simulador iOS executa o app compilado para o Mac, sem imitar o hardware.
6. Bluetooth, NFC e USB; medir desempenho e bateria; câmera e sensores reais.
7. Quando a mudança afeta o estado inicial (`main`, `initState`, variáveis globais) ou a estrutura de classes, que o *hot reload* não reaplica.
8. Reproduzir o fechamento, ler a pilha no console, colocar um *breakpoint* antes da linha suspeita, testar a hipótese, corrigir e criar um teste.

Fim do Módulo 01

Próximo: Módulo 02 — Dart essencial: tipos, null safety, funções, classes, coleções, records e padrões.